8-Bit Multi-Cycle CPU in Verilog — Project Requirement Plan

Project Goal
Design, implement, verify, and document a modular 8-bit multi-cycle CPU using Verilog HDL. The project must demonstrate RTL design, FSM-based control, datapath integration, memory operations, and verification methodology. The architecture should be scalable for future migration to a pipelined RISC-V CPU.

Category	Requirement
Architecture	8-bit multi-cycle CPU with FSM controller
Instruction Set	Arithmetic, logic, load/store, branch, jump
Registers	8 general-purpose registers
Memory	256x8 instruction memory and data memory
ALU	ADD, SUB, AND, OR, XOR, shift operations
Control Unit	Finite State Machine-based execution control
Verification	Self-checking Verilog testbench
Simulation	ModelSim/QuestaSim waveform verification
Documentation	Professional GitHub README and architecture diagrams

Core Deliverables
• Verilog RTL modules for all CPU blocks
• Modular datapath and control-path integration
• Self-checking testbench with pass/fail reporting
• Instruction memory initialization file
• Waveform screenshots for major instructions
• CPU architecture block diagram
• GitHub repository with structured folders
• Professional README with setup and simulation guide
• Final demonstration video (optional but recommended)

Development Milestones
1. ALU design and verification
2. Register file implementation
3. Program counter and instruction memory
4. Control unit FSM design
5. Datapath integration
6. CPU instruction execution validation
7. Full-system simulation and debugging
8. GitHub documentation and cleanup

Future Upgrade Path
• 16-bit / 32-bit architecture migration
• Pipelined CPU implementation
• Hazard detection and forwarding
• Cache memory integration
• UART and peripheral support
• FPGA deployment
• RISC-V ISA implementation

CPU Architecture Reference

PROJECT: 8-BIT MULTI-CYCLE CPU IN VERILOG

Design • RTL • Verification • Documentation • GitHub

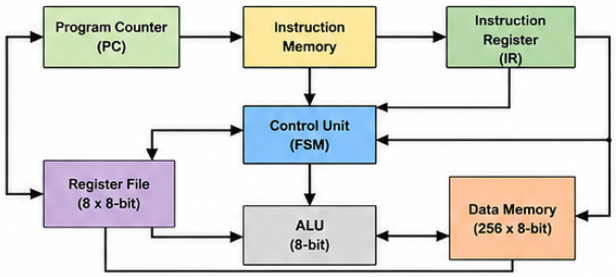
1. PROJECT OVERVIEW

Design and verify a complete 8-bit multi-cycle CPU in Verilog HDL. CPU supports a small instruction set, executes instructions using a multi-cycle FSM controller. Strong foundation for future upgrades to pipelined and RISC CPUs.

2. OBJECTIVES

- Build a working 8-bit CPU from scratch.
- Implement Datapath + Control path.
- Support instruction execution, memory access and branching.
- Verify with self-checking testbench.
- Provide clean, modular, well-documented code.
- Make project GitHub & resume ready.

3. CPU ARCHITECTURE



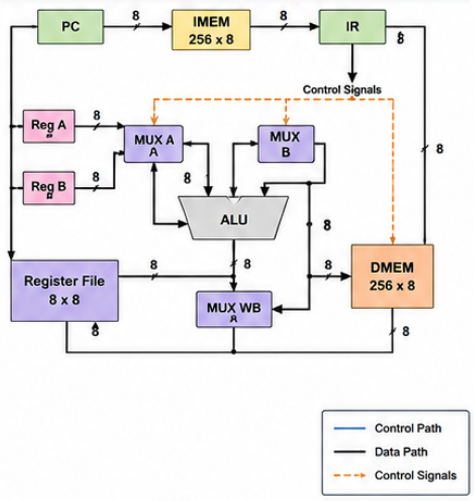
4. KEY FEATURES

- 8-bit data path
- 256 bytes Instruction Memory
- 256 bytes Data Memory
- 8 General Purpose Registers (R0–R7)
- Multi-cycle Execution (FSM based)
- Zero Flag (Z), Carry Flag (C)
- Load/Store, Arithmetic, Logic, Branch/Jump
- Easy to extend

5. INSTRUCTION SET (v1.0)

Opcode (4-bit)	Instruction	Format	Description
0000	NOP	-	No operation
0001	LOAD	Rd, [addr]	Load from memory to register
0010	STORE	Rs, [addr]	Store register to memory
0011	MOV	Rd, Rs	Copy register
0100	ADD	Rd, Rs	Rd = Rd + Rs
0101	SUB	Rd, Rs	Rd = Rd - Rs
0110	AND	Rd, Rs	Rd = Rd & Rs
0111	OR	Rd, Rs	Rd = Rd | Rs
1000	XOR	Rd, Rs	Rd = Rd ^ Rs
1001	ADDI	Rd, imm	Rd = Rd + imm
1010	JMP	addr	Jump to address
1011	JZ	addr	Jump if Zero flag = 1
1100	JC	addr	Jump if Carry flag = 1
1111	HLT	-	Halt CPU

6. CPU DATAPATH (FIGURE)



7. CPU OPERATION (MULTI-CYCLE FSM)

State	Operation
FETCH	PC → IMEM, IR ← IMEM[PC], PC ← PC + 1
DECODE	Decode IR, Read registers
EXECUTE	ALU operation / Address calculation
MEMORY	Read/Write Data Memory (if required)
WRITEBACK	Write result to Register File
HALT	Stop execution

8. DELIVERABLES

- [x] Complete Verilog RTL source code
- [x] Testbench with self-checking (pass/fail)
- [x] Waveforms for all instructions
- [x] Instruction set documentation
- [x] Architecture diagrams (Datapath + Control)
- [x] User guide (How to run, test, extend)
- [x] GitHub repository (clean & professional)
- [x] Demo video (optional but recommended)

9. VERIFICATION PLAN

1. Unit test each module (ALU, RegFile, PC, IMEM, DMEM, etc.)
2. Integrate and test control unit + datapath
3. Program test:
   - Arithmetic operations
   - Logical operations
   - Memory read/write
   - Branching and jumps
   - Flag conditions
   - Random program test
4. Functional coverage (all opcodes)
5. Corner cases and reset behavior

10. PROJECT STRUCTURE (GITHUB)

```
/cpu_8bit_multicycle
├── rtl/
│   ├── alu.v
│   ├── reg_file.v
│   ├── pc.v
│   ├── imem.v
│   ├── dmem.v
│   ├── control_unit.v
│   ├── cpu_top.v
│   └── fsm.v
├── tb/
│   └── tb_cpu.v
├── docs/
│   ├── architecture.png
│   └── instruction_set.md
├── README.md
└── LICENSE
```

11. FUTURE UPGRADES

- 16-bit data path
- Larger register file
- Pipelined CPU
- Hazard detection
- Cache
- Interrupts
- RISC-V ISA (next big step)

12. SUCCESS CRITERIA

- [x] All instructions execute correctly
- [x] Testbench 100% pass
- [x] Clean, modular, and documented code
- [x] Easy to understand and extend
- [x] Strong GitHub presentation

★ WHY THIS PROJECT IS STRONG

- Covers core Computer Architecture concepts
- Full RTL + Verification (Industry standard flow)
- Great for Resume and Interviews
- Foundation for RISC CPU and SoC projects

PROJECT TITLE SUGGESTION

"Design and Verification of an 8-bit Multi-Cycle CPU in Verilog"

TECH STACK

Verilog • ModelSim/QuestaSim
Vivado • GitHub